

Generative machine learning models for scenario simulation

IA pour la Science

4 may 2026

Generate a scenario with a probabilistic model

Terms:

- **scenario** is a time series realization of a reconstructed stochastic process,
- **machine learning** selects a reconstruction model to fit data,
- model **generates** data of same distribution.

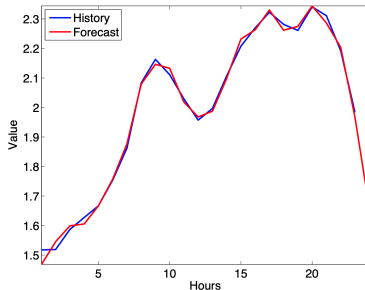
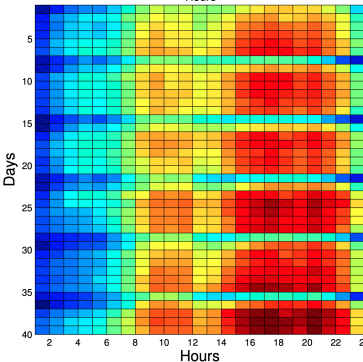
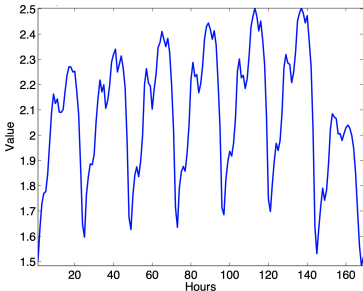
The goal: to reconstruct true distribution of data

The method: Principal Component Analysis (as autoencoder)

Assumptions:

- ① time series are relatively short,
- ② variance of each time series is high,
- ③ covariance of some time series is high.

One-state scenario forecasting model



The design matrix is

$$\begin{array}{c|c} y_{t+1} & \mathbf{x}_t \\ \hline 1 \times 1 & 1 \times n \\ \hline \mathbf{y} & \mathbf{X} \\ t \times 1 & t \times n \end{array},$$

the forecasting model $\hat{\mathbf{y}} = \mathbf{w}^\top \mathbf{X}$,
the forecast $\hat{y}_{t+1} = \mathbf{w}^\top \mathbf{x}_t$.

Singular Spectrum Analysis and state space

- ① Construct the Hankel matrix with time series,

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \vdots \\ \mathbf{x}_t \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 & \dots & x_n \\ x_2 & x_3 & x_4 & \dots & x_{n+1} \\ x_3 & x_4 & x_5 & \dots & x_{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_t & x_{t+1} & x_{t+2} & \dots & x_{t+n} \end{bmatrix}.$$

- ② Decompose the matrix and take the k first components,

$$\mathbf{X} = \mathbf{X}_1 + \dots + \mathbf{X}_n = \mathbf{U} \mathbf{\Lambda} \mathbf{V}^T = \sum_{k=1}^n \lambda_k \mathbf{u}_k \mathbf{v}_k^T.$$

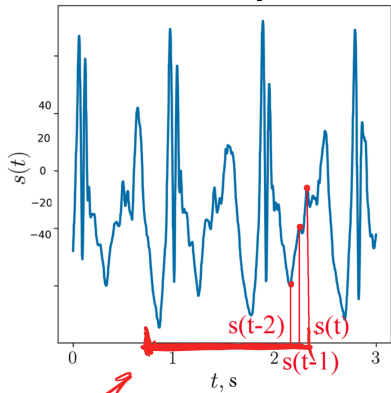
- ③ Reconstruct the time series by anti-diagonal average

$$\hat{x}_t = \text{mean} \mathbf{X}(i, j), \quad i + j = t - 1.$$

State space models describe the change of state $\mathbf{x}$ in time $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{z})$

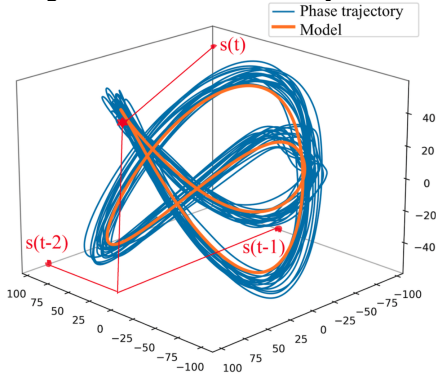
Define the state space by vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_t\}$, the phase trajectory $\mathbf{X}$.

Model of the phase trajectory in the state space



$t - 1000$

$\dim(s) \approx 1000$



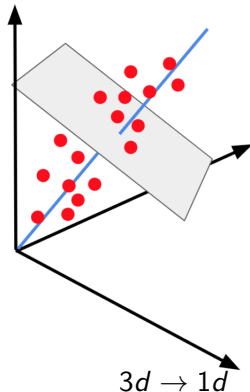
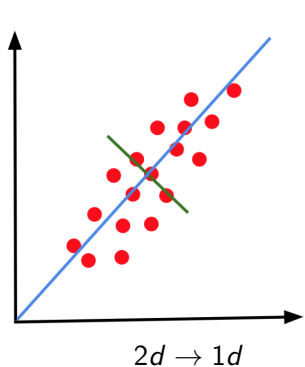
$\dim(x) = 4$

Reduce dimensionality with the principal component analysis, autoencoder $\mathbf{z} = \mathbf{W}^T \mathbf{x}$ where $\mathbf{W}$ is an orthogonal (rotation) matrix. The first principal components are given by Singular Values Decomposition

$$\sqrt{\lambda_k} \mathbf{V}_k = \mathbf{X}^T \mathbf{U}_k \quad \text{the SVD is} \quad \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{V}^T$$

Principal component analysis

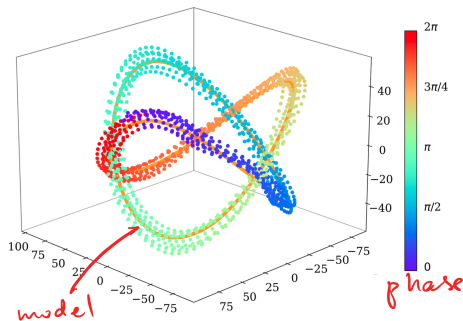
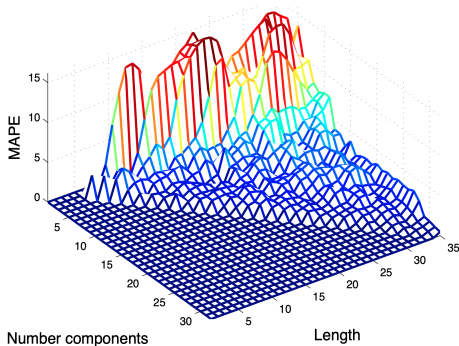
PCA reconstructs dependency, reduces dimensionality



$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}$$

PCA decomposes a set into deterministic $\mathbf{W}, \boldsymbol{\mu}$ and stochastic $\boldsymbol{\varepsilon}$ parts.

The model complexity and the phase trajectory

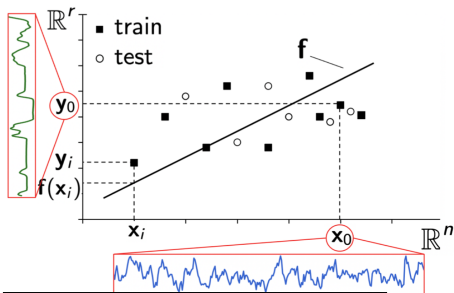
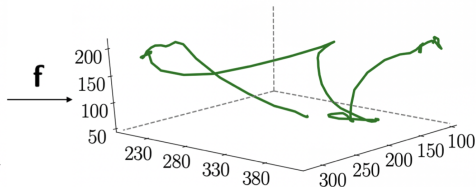
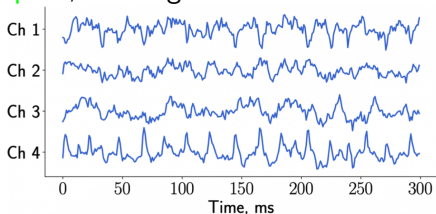


The length is n , here a point of the phase trajectory $\mathbf{x}_t \in \mathbb{R}^n$, the complexity is k . The encoder is

$$\begin{matrix} \mathbf{z} & = & \mathbf{W}^T \mathbf{x} \\ 1 \times k & & k \times n \quad n \times 1 \end{matrix}$$

Canonical correlation analysis

To control the scenario, it reconstructs dependencies in **design space**, **target space**, and align in-between. The forecasting model is $f = W\Lambda Q$.

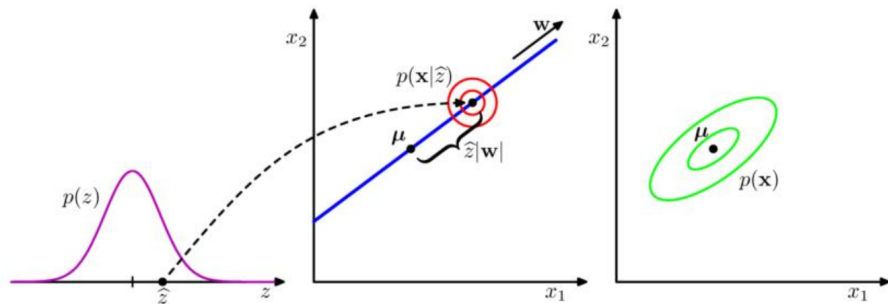


$$\begin{array}{ccc}
 \mathbf{x} \in \mathbb{R}^n & \xrightarrow{\mathbf{f}} & \mathbf{y} \in \mathbb{R}^r \\
 \swarrow \mathbf{W} & & \searrow \mathbf{C} \\
 & \mathbf{t}, \mathbf{u} \in \mathbb{R}^\ell & \\
 \nwarrow \mathbf{P} & & \nearrow \mathbf{Q} \\
 \mathbf{x} = \mathbf{P}\mathbf{t} + \mathbf{e}_x & & \mathbf{y} = \mathbf{Q}\mathbf{u} + \mathbf{e}_y \\
 \text{cov}(\mathbf{t}, \mathbf{u}) \rightarrow \max_{\mathbf{P}, \mathbf{Q}} & &
 \end{array}$$

Quadratic programming feature selection for multicorrelated signal decoding by
R. Isachenko and V. Strijov, 2022, Expert Systems

The simplest generative model

Probabilistic principal component analysis: to reconstruct the initial data, sample from the normal distribution

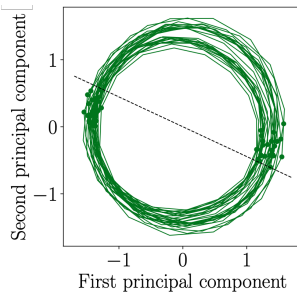
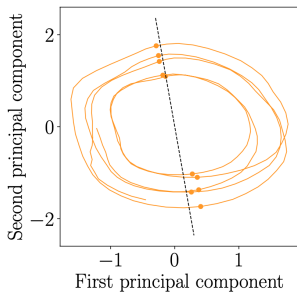
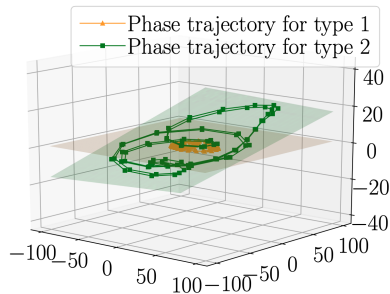
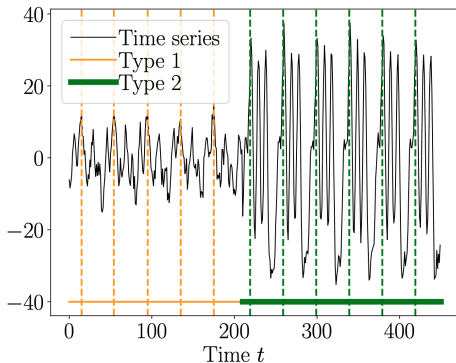


$$\mathbf{x} = \underbrace{\mathbf{W}^T \mathbf{z} + \boldsymbol{\mu}}_{\text{deterministic}} + \underbrace{\boldsymbol{\varepsilon}}_{\text{stochastic}} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}).$$

Denote by $p(z)$ distribution in latent space, and $p(\mathbf{x} | \mathbf{z})$ in the data space given the latent variable $\mathbf{z}$.

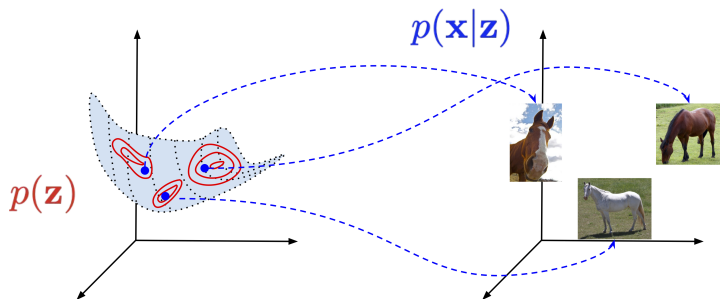
A latent space-based estimation of distribution algorithm for large-scale global optimization by W. Dong et al., 2018, Soft Computing

Select an optimal manifold, given a mixture



The simplest generative model, a mixture

Each data cluster has its own mean and variance.

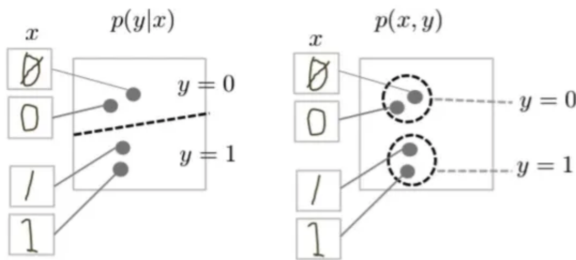


PCA reveals manifolds and reduces data dimensionality. Shall we use a deterministic or a probabilistic manifold?

Generative versus discriminative models

The variable $\mathbf{x}$ is either probabilistic or deterministic.

- Discriminative Model
- Generative Model



The Bayes' rule

$$p(y | \mathbf{x}) = \frac{\overbrace{p(\mathbf{x} | y)p(y)}^{p(\mathbf{x},y)}}{p(\mathbf{x})}$$

Fig 1. From a Machine Learning course by [Google](#)

Discriminative: in the logistic regression $\mathbf{x}$ is not a random variable,

$$p(y | \mathbf{x}) = (1 + \exp(-\mathbf{w}^T \mathbf{x}))^{-1}.$$

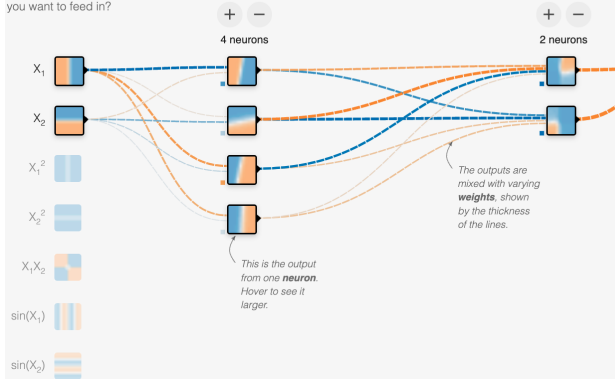
Generative: in the naive Bayesian $\mathbf{x}$ is a random variable, here it is normally distributed,

$$p(\mathbf{x} | y_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp -\frac{1}{2\sigma_k^2}(\mathbf{x} - \mathbf{c}_k)^2.$$

Non-linearity via linear maps and link functions

FEATURES

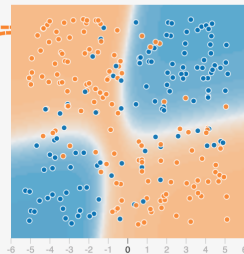
Which properties do you want to feed in?



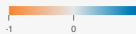
OUTPUT

Test loss 0.273

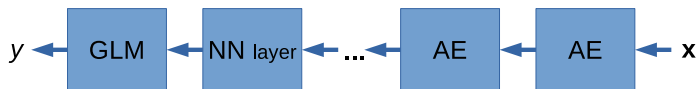
Training loss 0.251



Colors shows data, neuron and weight values.



Neural network with stack of autoencoders



$$f = \mathbf{w}_{1 \times 1_k}^T \mathbf{z}_{k-1} \circ \mathbf{W}_{k-1}^T \mathbf{z}_{k-2} \circ \cdots \circ \mathbf{W}_2^T \mathbf{z}_1 \circ \mathbf{W}_1^T \mathbf{x}$$

$n_2 \times 1 \quad n_1 \times n \quad n \times 1$

Neural network error

$$E_y = (y_i - f(\mathbf{x}))^2$$

Autoencoder reconstruction error

$$E_x = \|\mathbf{x} - \mathbf{r}(\mathbf{z})\|^2$$

Types of autoencoders

PCA
 $\mathbf{W}^T \mathbf{W} = \mathbf{I}_n$

skip block
 $\mathbf{W} = \mathbf{I}_n$

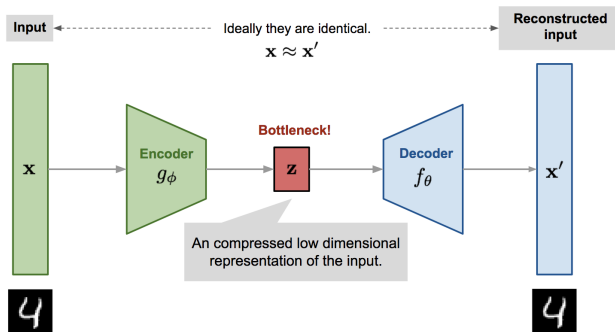
metric
 $\mathbf{x}^T \mathbf{W} \mathbf{x} \geq 0$

multi-linear
 $\mathbf{W} \mathbf{X}$

Autoencoder transform: $\mathbf{z} = (1 + \exp(-\mathbf{W}^T \mathbf{x} + \mathbf{b}))^{-1}$

Reconstruction decoder: $\hat{\mathbf{x}} = \mathbf{r}(\mathbf{z}(\mathbf{x}))$

Autoencoder: probabilistic or deterministic?

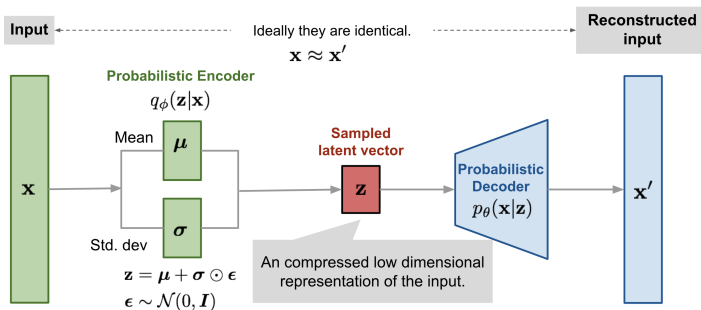


Encoder g_ϕ translates the original high-dimensional $\mathbf{x}$ to the low-dimensional latent $\mathbf{z}$.

Decoder f_θ reconstructs original $\hat{\mathbf{x}} \sim \mathbf{x}$ with the loss function

$$\left(\mathbf{x} - f_\theta \left(\underbrace{g_\phi(\mathbf{x})}_{\text{low-dim latent } \mathbf{z}} \right) \right)^2$$

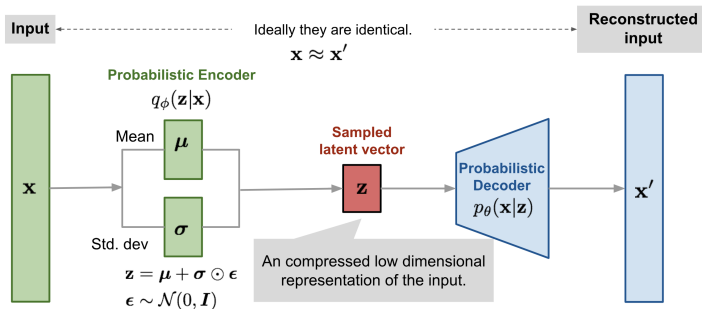
Variational autoencoder



Encoder $q_{\phi}(\mathbf{z} | \mathbf{x}) = \text{NN}_{\text{enc}}(\mathbf{x}, \phi)$ outputs $\mu_{\phi}(\mathbf{x})$ and $\sigma_{\phi}(\mathbf{x})$.

Decoder $p_{\theta}(\mathbf{x} | \mathbf{z}) = \text{NN}_{\text{dec}}(\mathbf{z}, \theta)$ outputs $\mathbf{x}$ of similar distribution.

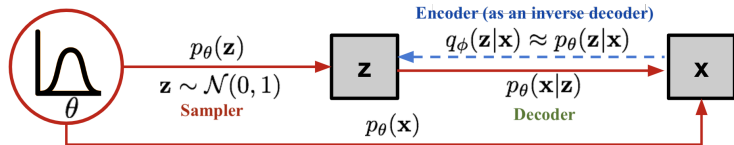
Variational autoencoder



To generate a sample $\mathbf{x}_i$ that **looks like real data**

- ① sample $\mathbf{z}_i$ from the prior distribution $p_{\hat{\theta}}(\mathbf{z})$
- ② then generate $\mathbf{x}_i$ from the conditional distribution $p_{\hat{\theta}}(\mathbf{x} | \mathbf{z} = \mathbf{z}_i)$

Graphical model of the Variational autoencoder



Approximate the probability to generate real-data samples with $q_\phi(z | x)$

$$\hat{\theta} = \arg \max \sum_{i=1}^m \log p_\theta(\mathbf{x}_i), \text{ where } p_\theta(\mathbf{x}_i) = \int p_\theta(\mathbf{x}_i | z) p_\theta(z) dz$$

Loss function to teach the network parameters

$$L_{\text{VAE}}(\phi, \theta) = \log p_\theta(\mathbf{x}) + \underbrace{D_{\text{KL}}(q_\phi(z | \mathbf{x}) \| p_\theta(z | \mathbf{x}))}_{\text{distributions look similar}} =$$

$$-E_{z \sim q_\phi(z|x)} \log p_\theta(\mathbf{x} | z) + D_{\text{KL}}(q_\phi(z | \mathbf{x}) \| p_\theta(z)),$$

$$\hat{\phi}, \hat{\theta} = \arg \min L_{\text{VAE}}$$

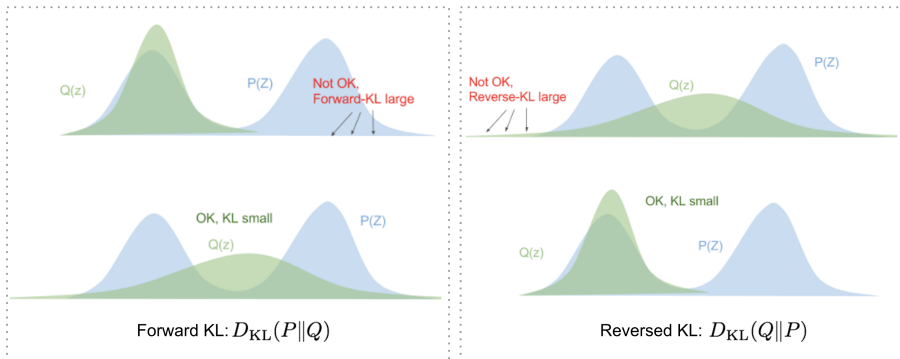
Design of Variational Autoencoder for Generation by A., Das et al., 2021, Innovation in Power Eng.

Multi-modal distribution of data and uni-modal prior

Forward and reversed KL divergence

$$D_{\text{KL}}(p \parallel q) = \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} dx$$

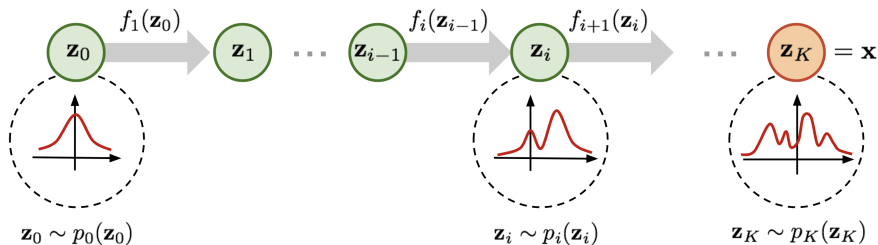
matches two distributions in different ways.



The normalizing flow is a superposition

The flow $f_1, \dots, f_K$ must be

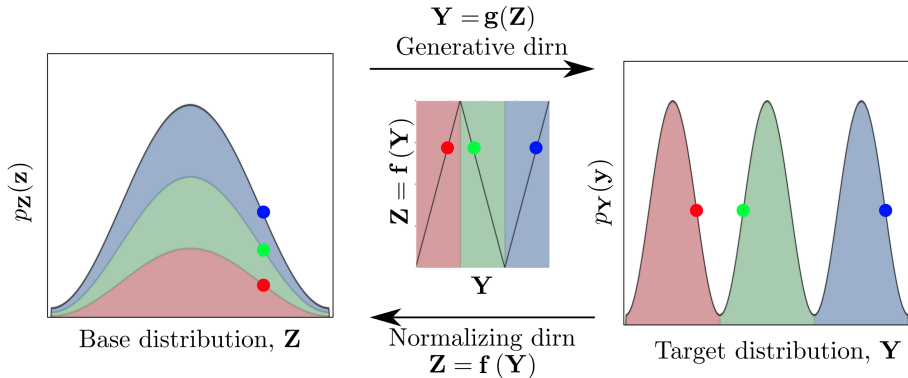
- ① easily invertible,
- ② its Jacobian determinant is easy to compute.



The target distribution $\log p_K(\mathbf{z}_K) = \log p_0(\mathbf{z}_0) - \sum_{i=1}^K \log \left(\det \frac{df_i(\mathbf{z}_{i-1})}{d\mathbf{z}_{i-1}} \right)$.

Let $\mathbf{z}_0 = f_1^{-1}(\mathbf{z}_1)$. Change variables in the pdf so $p_1(\mathbf{z}_1) = p_0(\mathbf{z}_0) \left(\det \frac{df_1^{-1}(\mathbf{z}_1)}{d\mathbf{z}_1} \right)$.

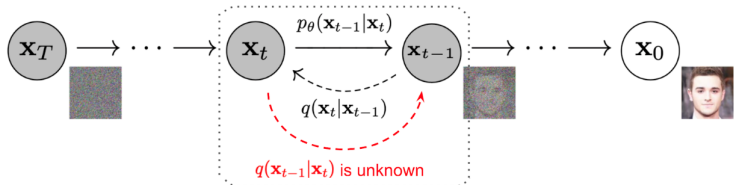
An example of the flow: piecewise bijective



A monotone function maps sections of data domain to the base distribution.

To invert the function, sample the base distribution with a gating network. Use a mixture of experts network.

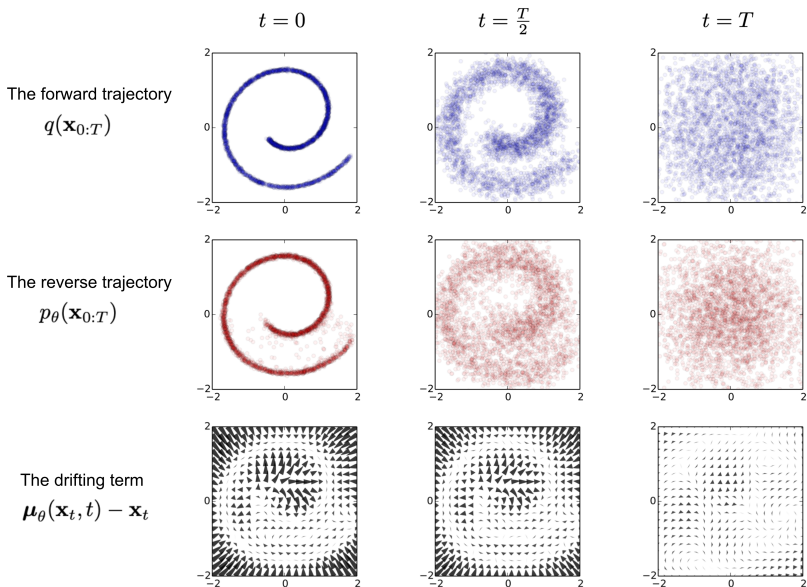
Diffusion models: learn slowly by adding noise



Given the data distribution $\mathbf{x}_0 \sim q(\mathbf{x})$ set:

- ① forward diffusion process $\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathbf{z}_t$, sampling i.i.d. $\mathbf{z}_1, \dots, \mathbf{z}_T \sim \mathcal{N}(0, \mathbf{I})$,
- ② sampling $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ and learning parameters θ of U-Net $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$,
- ③ reverse diffusion process $p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$,
- ④ slow learning gives $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t))$.

An example of training a diffusion model



Building complex generative models

The main challenge is to estimate the normalizing constant

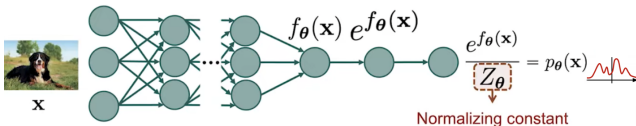
$$Z_{\theta} = \int \exp(f_{\theta}(\mathbf{x})) d\mathbf{x}$$



Unknown data
distribution



Model distribution

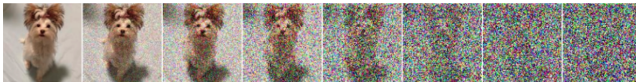


Approximate unknown true distribution with a neural
network

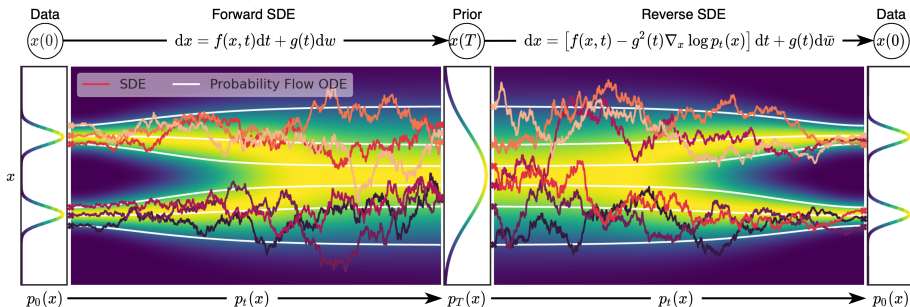
Normalizing for Gaussian distribution $Z_{\mu} = 2\pi^{-\frac{d}{2}}$

Score-based generative model via Neural SDEs

SDE smoothly transforms a complex data distribution to a known prior distribution by slowly injecting noise.



To reverse the SDE compute the score $\nabla_x \log p_t(x)$ of the distribution at each t .



Score-based generative modeling through SDE by Y. Song, 2022, ArXiv

Create a test generative model

A 30-second project. Request for a code

 You
python, create a simple variational autoencoder and generate periodic time-series

 ChatGPT

Define the model

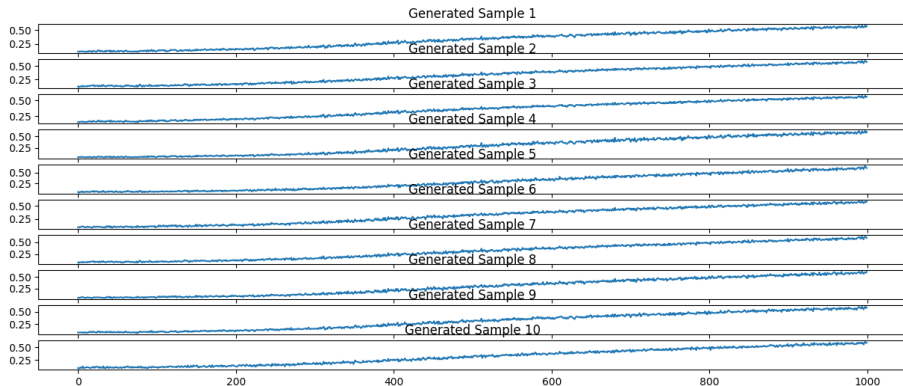
```
class VAE(models.Model):  
    def __init__(self, encoder, decoder, **kwargs):  
        super(VAE, self).__init__(**kwargs)  
        self.encoder = encoder  
        self.decoder = decoder  
        self.total_loss_tracker = tf.keras.metrics.Mean(name="total_loss")  
        self.reconstruction_loss_tracker = tf.keras.metrics.Mean(  
            name="reconstruction_loss"  
        )  
        self.kl_loss_tracker = tf.keras.metrics.Mean(name="kl_loss")
```

Run with a useful optimizer

```
# Instantiate the VAE model  
vae = VAE(encoder, decoder)  
vae.compile(optimizer=tf.keras.optimizers.Adam())
```

This code is generated by ChatGPT 3.5

The generated time series



This code is generated by ChatGPT 3.5

PyTorch: Variational Autoencoder example

```
class Encoder(nn.Module):
    def __init__(self, latent_dims):
        super(Encoder, self).__init__()
        self.linear1 = nn.Linear(784, 512)
        self.linear2 = nn.Linear(512, latent_dims)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.linear1(x))
        return self.linear2(x)
```

```
class Decoder(nn.Module):
    def __init__(self, latent_dims):
        super(Decoder, self).__init__()
        self.linear1 = nn.Linear(latent_dims, 512)
        self.linear2 = nn.Linear(512, 784)

    def forward(self, z):
        z = F.relu(self.linear1(z))
        z = torch.sigmoid(self.linear2(z))
        return z.reshape((-1, 1, 28, 28))
```


PyTorch: Variational Autoencoder example

```
class Autoencoder(nn.Module):
    def __init__(self, latent_dims):
        super(Autoencoder, self).__init__()
        self.encoder = Encoder(latent_dims)
        self.decoder = Decoder(latent_dims)

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

def train(autoencoder, data, epochs=20):
    opt = torch.optim.Adam(autoencoder.parameters())
    for epoch in range(epochs):
        for x, y in data:
            x = x.to(device) # GPU
            opt.zero_grad()
            x_hat = autoencoder(x)
            loss = ((x - x_hat)**2).sum()
            loss.backward()
            opt.step()
    return autoencoder

autoencoder = train(autoencoder, data)

z = torch.Tensor([[x, y]]).to(device)
x_hat = autoencoder.decoder(z)
```

To program a simple project

Set an object to sample

describe a type of relations between time series (none, multi-linear, metric)

- ① a point in a phase trajectory (a tensor in a set of trajectories)
- ② a dynamic graph in a graph trajectory
- ③ a couple of CCA source and target trajectories

Set a generative model to tune

put forward a hypothesis on data distribution; it makes optimization criteria to tune NN

- ① variational auto-encoder
- ② normalizing flow model
- ③ diffusion probabilistic model

Set an external utility function

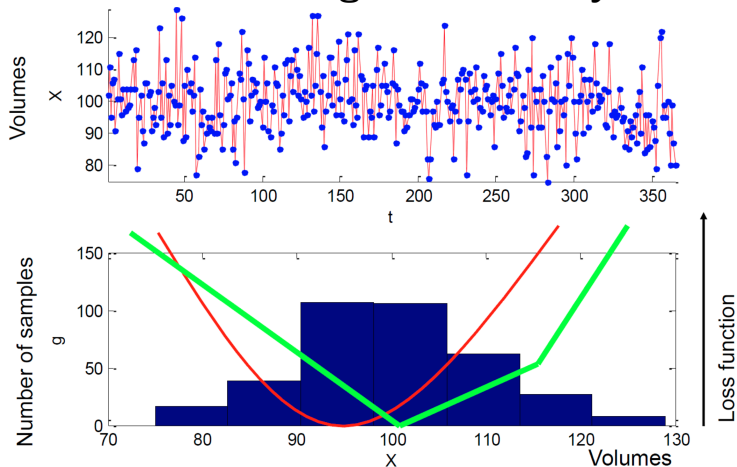
it selects a type of model and structure of the neural network

Select performance measurement routine and dataset

Use benefits of generative models

- ① Optimal scenario analysis and selection using convolution model
- ② Uncertainty and out-of-distribution detection to avoid unstable decisions
- ③ Transfer Learning, Multi-Modal, and Capturing Long-Term Dependencies

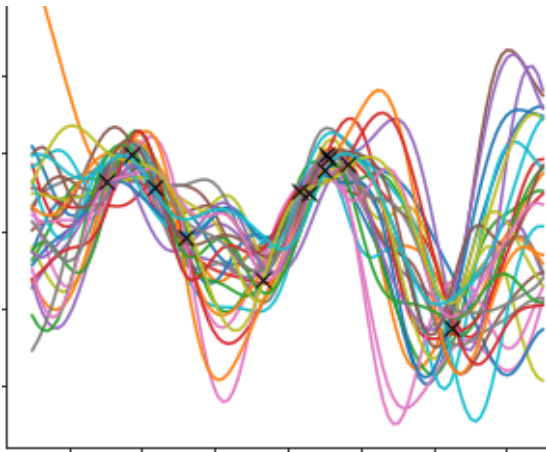
Convolution with an engineered utility function



There given the histogram $\{x_i, g_i\}$ and the utility function $L(z, x)$, for example, $|z - x|$ or $(z - x)^2$. Find the forecast $\hat{x}$ as

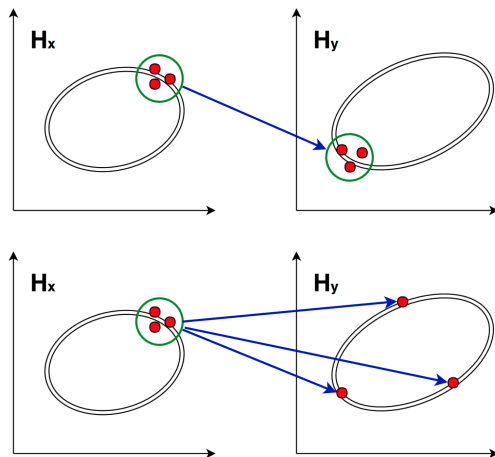
$$\hat{x} = \arg \min_{z \in \{x_1, \dots, x_m\}} \sum_{i=1}^m g_i L(z, x_i).$$

Uncertainty and out-of-distribution detection



Avoid unstable decisions in the presence of lack of data or new situation

Convergent cross mapping as a distance function

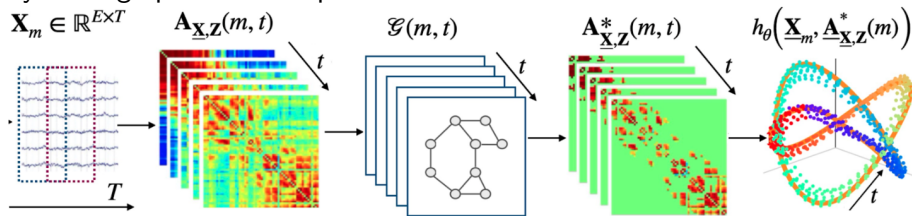


The time series $\mathbf{y}$ depends on the time series $\mathbf{x}$, if in the neighbourhood $(\mathbf{x}, \mathbf{x}') \in \mathbf{H}_x$ there exists a **Lipschitz continuous map**

$$\varphi: \mathbf{H}_x \rightarrow \mathbf{H}_y \quad \text{such that} \quad \rho_{\mathbf{H}_y}(\varphi(\mathbf{x}, \mathbf{x}')) \geq L \rho_{\mathbf{H}_x}(\mathbf{x}, \mathbf{x}').$$

High variance and high co-variance in time series

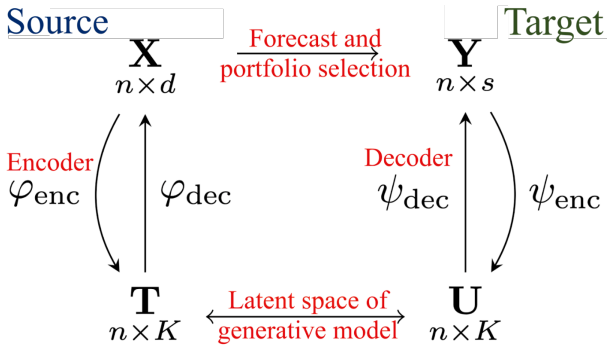
Dynamic graph reflects dependencies between the time series.



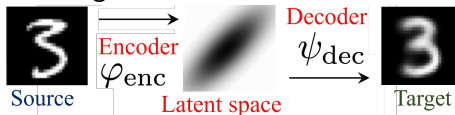
To reconstruct the dependencies

- ① define distance between points of the phase trajectories,
- ② make low-rank decomposition, prune the dependency graph,
- ③ reconstruct time series.

Generative model for Canonical Correlation Analysis



- ① Approximates both spaces, design and target
- ② Reduce the dimensionality of spaces, select the connected data
- ③ Select a subset of target time series



Tools to create generative models

General purpose

- ① PyTorch, TensorFlow (Keras, TFP), and JAX
- ② DCGAN, Torch-GAN and Conditional GAN
- ③ TabbyML, Copilot, Google AutoML

Generative models and collections

- ① Pytae: most common variational autoencoder models
- ② UNet diffusion: denoising diffusion probabilistic model in PyTorch
- ③ PGMC: collection of generative models in PyTorch

CCA and Graph networks

- ① DeepCCA: deep canonical correlation analysis
- ② DGCCA: deep generalized canonical correlation analysis
- ③ pyRiemann: Biosignals classification with Riemannian geometry

Articles to read

- ① Introduction to Probabilistic Programming by A. Das, 2020, ayandas
- ② Foundation of Variational Autoencoder by A. Das, 2020, ayandas
- ③ From Autoencoder to Beta-VAE by L. Weng, 2018, GitHub
- ④ Flow-based Deep Generative Models by L. Weng, 2018, GitHub
- ⑤ Normalizing Flows: review by I. Kobyzin et al., 2020, IEEE
- ⑥ Variational Inference with Normalizing Flows by D.J. Rezende, S. Mohamed, 2015, ArXiv
- ⑦ Score-Based Generative Modeling through Stochastic Differential Equations by Y. Song et al., 2015, ArXiv
- ⑧ Denoising diffusion probabilistic models by J. Ho, 2020 ArXiv
- ⑨ Deep unsupervised learning using Nonequilibrium Thermodynamics by J. Sohl-Dickstein et al., 2015, ArXiv
- ⑩ An Intuitive Tutorial to Gaussian Processes Regression by J. Wang, 2020, ArXiv